End-to-End Data Management Pipeline for a Movie Recommender

Data Management for Machine Learning, Assignment 2

Harshini B (2025AG05439) | Repository: https://github.com/BBHarshini/recommendation-pipeline

1. Overview

The goal was to build a pipeline that takes raw user, movie and rating data and turns it into evaluated recommendation models, with every step automated and repeatable. I used the public MovieLens 100K dataset: 100,000 ratings (1 to 5 stars) from 943 users on 1,682 movies, collected between September 1997 and April 1998. It contains all three required tables: user details (age, gender, occupation), item details (title, release date, genres) and timestamped ratings.

The pipeline has seven stages, each in its own module under src/, run in order by a Prefect flow. All settings (paths, validation rules, split size, model parameters and a fixed seed) live in one file, configs/config.yaml.

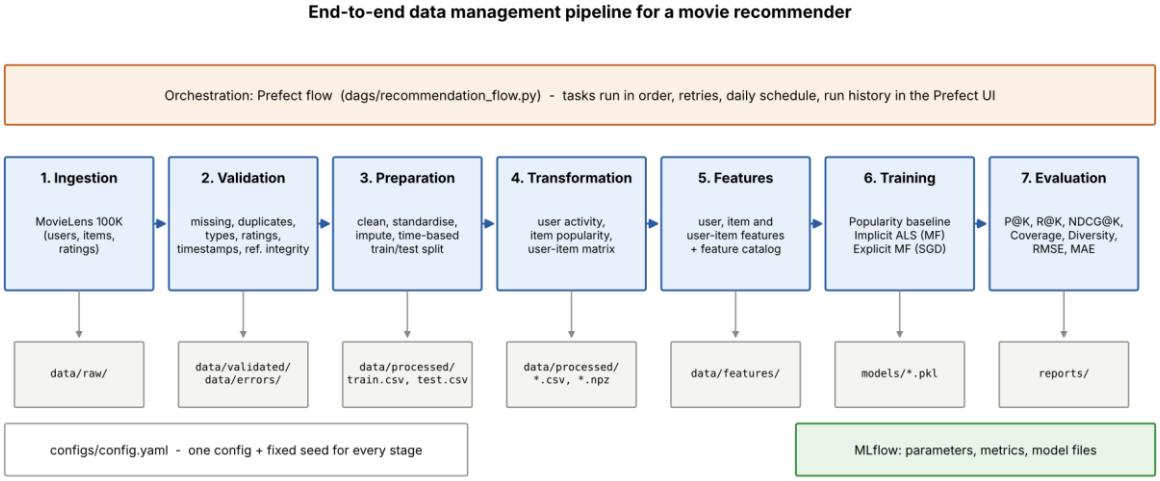


Figure 1. Pipeline architecture

2. Design decisions

Ingestion. The zip file is downloaded once and its three files are stored unchanged in data/raw. A manifest records the source, row counts and a SHA-256 checksum of each table, so it is possible to check later that the same data was used. The 19 one-hot genre columns are combined into one readable genres field.

Validation. Seventeen row-level checks cover missing values, duplicates, wrong data types, invalid ratings (not a whole number from 1 to 5), invalid timestamps (unparseable, before 1995 or in the future), age and gender rules, and referential integrity (every rating must point to a valid user and movie). Rows that fail an error-level check are moved to data/errors with the names of the failed checks; warnings are kept and fixed in preparation. Because MovieLens is already clean, ingestion appends 19 clearly labelled broken records (for example a rating of 7, the rating "five", a timestamp in 2035 and a rating for user 99999) so that every check can be seen working. All 19 were caught and no real MovieLens row was rejected. One movie had no release date and was flagged as a warning.

Preparation and transformation. Formats were standardised (types, upper/lower case, ISO dates), the missing release year was filled with the median and flagged, and repeated ratings of the same movie would keep only the latest one (none occurred). For the split I used a single global time cutoff: the newest 20% of ratings, from 7 March 1998, form the test set (79,999 training and 20,001 test ratings). A random split would let the model learn from ratings made after the ones it is tested on, which cannot happen in a real system. From the training data only, the transformation stage builds a user activity summary, an item popularity table and a sparse 751 x 1,616 user-item rating matrix (the users and movies seen in the training period; 6.6% of cells filled).

Feature management. Nineteen features are defined once in a feature registry (src/features/registry.py): nine user features (e.g. average rating, activity count, preferred genre, days since last rating), seven item features (e.g. popularity, Bayesian average rating, release year) and three user-item features (genre affinity, rating deviation, recency weight). Each feature is computed relative to a reference time, which is the split cutoff for training, so no test information leaks in; the same functions can be run with the current time for live predictions. The registry also marks features that use the rating itself as training-only. The feature catalogue (reports/feature_catalog.md) is generated from the registry, so documentation and code cannot drift apart. A small feature store class serves these features to the models.

Models. Three models were trained. The baseline recommends the most-rated movies a user has not yet seen. Implicit ALS is a matrix factorisation model that learns from which movies each user rated, using the star rating as a confidence weight (Hu, Koren and Volinsky, 2008). Explicit MF is a biased matrix factorisation model trained with stochastic gradient descent to predict the star rating itself. Movies with fewer than five training ratings are not recommended by any model.

Tracking, orchestration and reproducibility. Each model gets an MLflow run with its parameters, metrics, the saved model file and its top-10 lists. The Prefect flow runs the seven stages as tasks, where each task receives the previous task's result, so they always run in order and the flow stops if one fails; tasks retry once, and the flow can be served on a daily schedule. The complete run on MovieLens finished successfully in 1 minute 8 seconds. With the fixed seed, repeated runs give identical results. Eight unit tests check the validation rules, the metrics and the split.

3. Tools used

Purpose	Tool
Data handling and storage	Python, pandas, NumPy, SciPy sparse matrices; CSV files for tables, NPZ for the matrix
Data validation	Custom rule-based checks (src/validation/checks.py) with error datasets and a markdown/JSON report
Feature management	Custom feature registry and feature store
Models	Own NumPy implementations of popularity, implicit ALS and explicit matrix factorisation
Experiment tracking	MLflow (SQLite backend)
Orchestration	Prefect 3 (flow with seven tasks); run_pipeline.py as a plain-Python alternative
Testing and setup	pytest, uv, requirements.txt

4. Results

Evaluation used K = 10. A movie counts as relevant if the user rated it 4 or 5 stars in the test period. Ranking metrics are averaged over the 95 test users who also appear in training and have at least one relevant test movie. Coverage is the share of the catalogue that appears in at least one list, and diversity is the average genre dissimilarity (1 minus Jaccard similarity) between movies in the same list. RMSE and MAE use all 2,864 test ratings whose user and movie are known from training; the popularity model predicts each movie's Bayesian average rating.

Model	Prec.@10	Recall@10	NDCG@10	Coverage	Diversity	RMSE	MAE
Popularity (baseline)	0.1116	0.0877	0.1391	0.0470	0.8270	1.0276	0.8290
Implicit ALS	0.1074	0.0955	0.1339	0.2735	0.7899	n/a	n/a
Explicit MF	0.0621	0.0531	0.0838	0.1300	0.8014	0.9787	0.7682

2025AG05429 DMML Assignment2

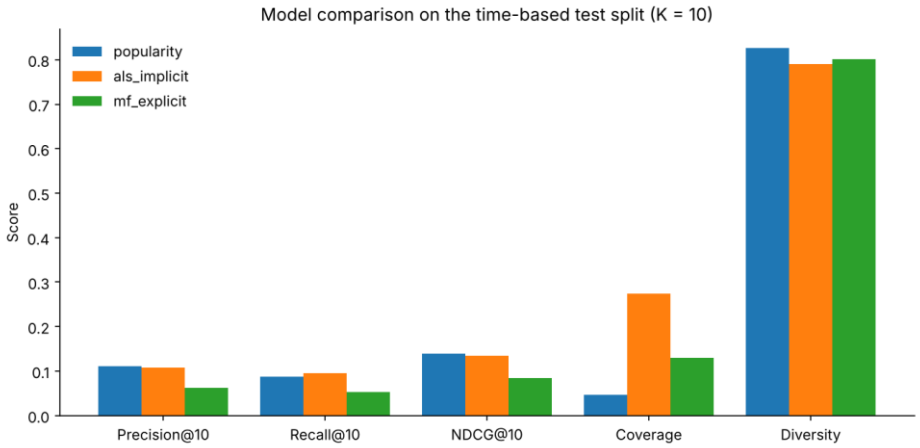


Figure 2. Ranking metrics, coverage and diversity (K = 10)

- **Popularity is a strong baseline.** It has the highest precision and NDCG, but it shows almost the same ten movies to everyone and uses only 4.7% of the catalogue.
- **Implicit ALS is the best personalised model.** Its precision and NDCG are within about 0.005 of the baseline, which is roughly four extra hits across all 95 users and well within chance at this sample size. It has the best recall and recommends 27% of the catalogue, about six times more than the baseline, so different users really do get different lists.
- **Explicit MF predicts ratings best but ranks worst.** Its RMSE is about 5% lower than the baseline's (0.979 vs 1.028), yet its top-10 lists are the weakest. It is trained to predict how many stars a user would give a movie, not whether the user will watch it, so it favours well-liked niche movies that most users never rate.
- Diversity is similar for all three models (0.79 to 0.83). The popularity list is slightly more varied in genre because the most-rated movies happen to span many genres.

5. Limitations and possible improvements

- **Small evaluation set.** The global time cutoff is the most realistic split, but 192 of the 301 test users had no ratings before the cutoff, so only 95 users could be evaluated and small differences are not reliable. Rolling evaluation over several cutoffs, or a larger dataset such as MovieLens 1M, would give firmer results.
- **Cold start.** New users cannot be served by the two matrix factorisation models. A production system would fall back to the popularity list or to profile features (age group, occupation) for them.
- **No hyperparameter tuning.** Model settings were chosen once and not tuned, because tuning on the test set would be leakage. A validation period carved from the end of the training data would allow fair tuning.
- **Validation evidence.** The dirty records used to demonstrate validation were injected, because the real data is clean. The checks themselves are general and would apply to any new data in the same format.
- **Storage and tooling.** CSV files and a custom feature store are enough at this size; Parquet or PostgreSQL storage and a feature store such as Feast would be needed for larger or continuously updated data. Diversity is measured on genres only.

6. Conclusion

The pipeline runs end to end with one command under Prefect, rejects invalid data with a clear record of why, keeps training and test data separated in time, and records every model run in MLflow. On MovieLens 100K, the popularity baseline was hard to beat on precision, implicit ALS matched it while covering six times more of the catalogue, and explicit MF showed that the best rating predictor is not necessarily the best recommender. Screenshots of every stage running are in Appendix A.

2025AC05439 DMML Assignment2

Appendix A. Execution screenshots

Screenshots taken while running the pipeline on MovieLens 100K on 7 November 2026 (Windows 11, Python 3.12), showing each stage executing and the output it produced. Code and outputs: https://github.com/BBHarshini/recommendation-pipeline

A.1 Complete pipeline run (all seven stages)

```
(.venv) PS C:\Users\DELL\Downloads\recommendation-pipeline\recommendation-pipeline> python run_pipeline.py
2026-11-07 15:22:21 | INFO     | === INGESTION ===
2026-11-07 15:22:21 | INFO     | Using cached download C:\Users\DELL\Downloads\recommendation-pipeline\recommendation-pipeline\data\raw\ml-100k.zip
2026-11-07 15:22:21 | INFO     | Loaded 943 users, 1682 items, 100000 interactions from 'movielens_100k'
2026-11-07 15:22:21 | INFO     | Appended labelled test records for validation (inject_test_errors = true)
2026-11-07 15:22:21 | INFO     | Raw tables written to C:\Users\DELL\Downloads\recommendation-pipeline\recommendation-pipeline\data\raw
2026-11-07 15:22:21 | INFO     | ingestion finished in 0.3 s -> {'users': 948, 'items': 1684, 'interactions': 100012}
2026-11-07 15:22:21 | INFO     | === VALIDATION ===
2026-11-07 15:22:22 | INFO     | Validation users       : 948 rows in, 943 valid, 5 rejected
2026-11-07 15:22:22 | INFO     | Validation items       : 1684 rows in, 1682 valid, 2 rejected
2026-11-07 15:22:22 | INFO     | Validation interactions: 100012 rows in, 100000 valid, 12 rejected
2026-11-07 15:22:22 | INFO     | validation finished in 0.7 s -> {'users': 5, 'items': 2, 'interactions': 12}
2026-11-07 15:22:22 | INFO     | === PREPARATION ===
2026-11-07 15:22:22 | INFO     | Cleaned data: 943 users, 1682 items, 100000 interactions (0 repeated ratings collapsed)
2026-11-07 15:22:23 | INFO     | Time split at 1998-03-07T02:21:09+00:00: 79999 train / 20001 test interactions; 109 of 301 test users have training history
2026-11-07 15:22:23 | INFO     | preparation finished in 1.3 s -> {'method': 'global time cutoff', 'cutoff_timestamp': 889237269, 'cutoff_datetime': '1998-03-07T02:21:09+00:00', 'train_interactions': 79999, 'test_interactions': 20001, '
train_period': ['1997-09-20 03:05:10+00:00', '1998-03-07 02:20:47+00:00'], 'test_period': ['1998-03-07 02:21:09+00:00', '1998-04-22 23:10:38+00:00'], 'test_users': 301, 'test_users_with_train_history': 109, 'test_users_new': 192, 'test
_interactions_on_items_unseen_in_train': 202, 'repeated_ratings_collapsed': 0, 'items_with_imputed_release_year': 1}
2026-11-07 15:22:23 | INFO     | === TRANSFORMATION ===
2026-11-07 15:22:23 | INFO     | User-item matrix 751 x 1616, 79999 ratings, density 6.59%
2026-11-07 15:22:23 | INFO     | transformation finished in 0.2 s -> {'matrix_shape': [751, 1616], 'density': 0.0659, 'users_summarised': 751, 'items_summarised': 1682}
2026-11-07 15:22:23 | INFO     | === FEATURES ===
2026-11-07 15:22:24 | INFO     | Built 19 features: 751 users, 1682 items, 79999 user-item rows
2026-11-07 15:22:24 | INFO     | features finished in 0.6 s -> {'features': 19, 'users': 751, 'items': 1682}
2026-11-07 15:22:24 | INFO     | === TRAINING ===
2026-11-07 15:22:25 | INFO     | Trained popularity    in    0.0 s (run cdc6040fd92d4a158901d7b8021396f7)
2026-11-07 15:22:28 | INFO     | Trained als_implicit  in    2.3 s (run 45705535a475430a83c4d68fb751770d)
2026-11-07 15:23:01 | INFO     | Trained mf_explicit   in   33.5 s (run 371acce73fbd451394a085d7d3f2c7d5)
2026-11-07 15:23:01 | INFO     | training finished in 37.6 s -> {'popularity': 0.0, 'als_implicit': 2.27, 'mf_explicit': 33.46}
2026-11-07 15:23:01 | INFO     | === EVALUATION ===
2026-11-07 15:23:02 | INFO     | Evaluating on 95 users (K = 10), 2864 test ratings for RMSE/MAE
2026-11-07 15:23:02 | INFO     | popularity    P@10=0.1116 R@10=0.0877 NDCG@10=0.1391 cov=0.047 div=0.827
2026-11-07 15:23:02 | INFO     | als_implicit  P@10=0.1074 R@10=0.0955 NDCG@10=0.1339 cov=0.274 div=0.790
2026-11-07 15:23:02 | INFO     | mf_explicit   P@10=0.0621 R@10=0.0531 NDCG@10=0.0838 cov=0.130 div=0.801
2026-11-07 15:23:02 | INFO     | evaluation finished in 0.6 s -> {'popularity': 0.1391, 'als_implicit': 0.1339, 'mf_explicit': 0.0838}
2026-11-07 15:23:02 | INFO     | Pipeline finished in 41.2 s
(.venv) PS C:\Users\DELL\Downloads\recommendation-pipeline\recommendation-pipeline>
```

Figure A1. Terminal output of python run_pipeline.py: every stage logs what it did; the full run takes 41 s

A.2 Stage 1: Data ingestion

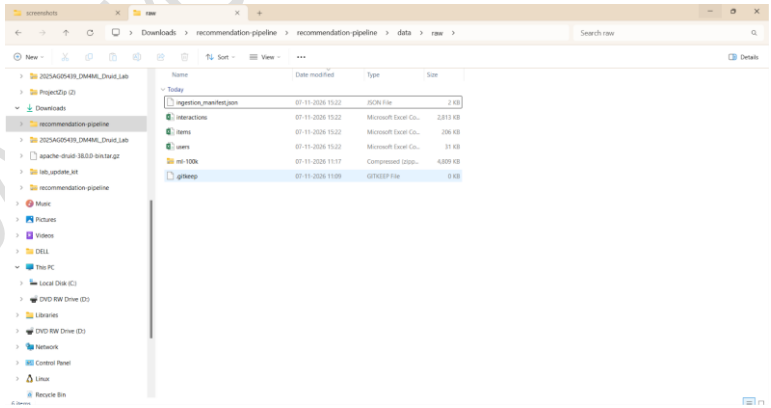


Figure A2. data/raw after ingestion: downloaded zip, the three raw tables and the ingestion manifest

A.3 Stage 2: Data validation

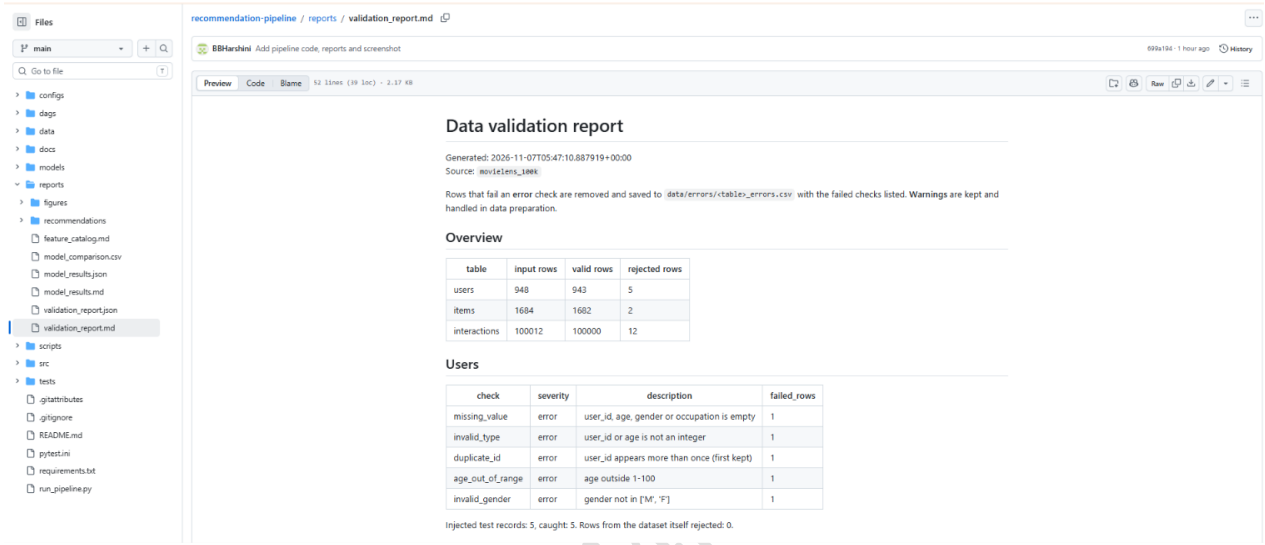


Figure A3. Validation report: overview and user checks

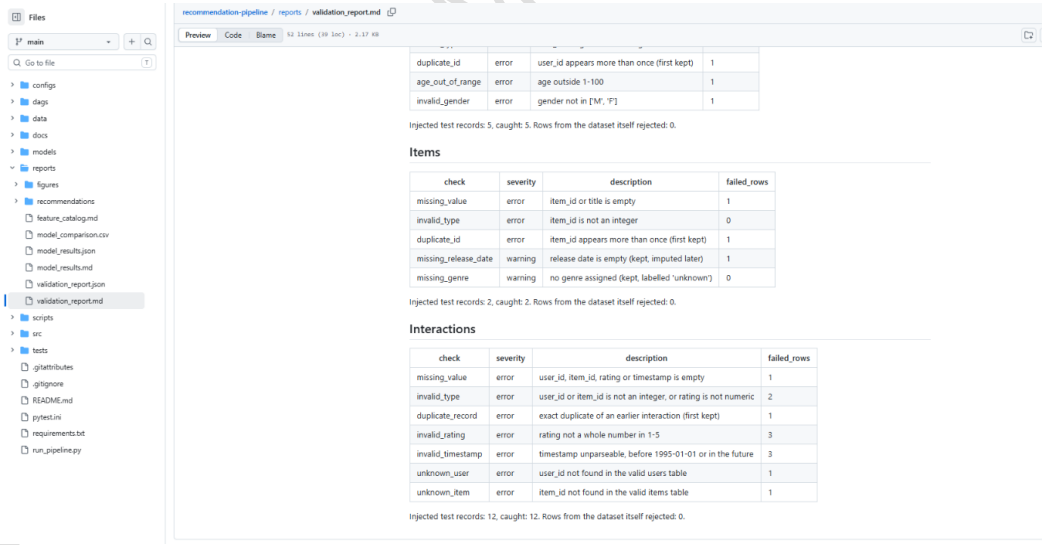
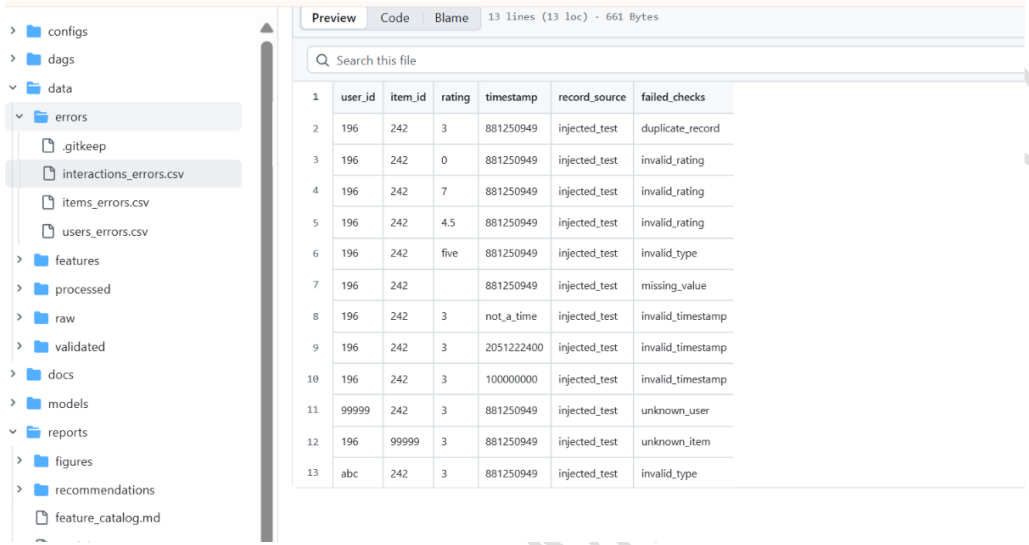


Figure A4. Validation report: item and interaction checks; all 19 injected errors caught, no real rows rejected



	user_id	item_id	rating	timestamp	record_source	failed_checks
2	196	242	3	881250949	injected_test	duplicate_record
3	196	242	0	881250949	injected_test	invalid_rating
4	196	242	7	881250949	injected_test	invalid_rating
5	196	242	4.5	881250949	injected_test	invalid_rating
6	196	242	five	881250949	injected_test	invalid_type
7	196	242		881250949	injected_test	missing_value
8	196	242	3	not_a_time	injected_test	invalid_timestamp
9	196	242	3	2051222400	injected_test	invalid_timestamp
10	196	242	3	100000000	injected_test	invalid_timestamp
11	99999	242	3	881250949	injected_test	unknown_user
12	196	99999	3	881250949	injected_test	unknown_item
13	abc	242	3	881250949	injected_test	invalid_type

Figure A5. Error dataset data/errors/interactions_errors.csv: each rejected row with the check it failed

A.4 Stage 3: Data preparation (time-based split)

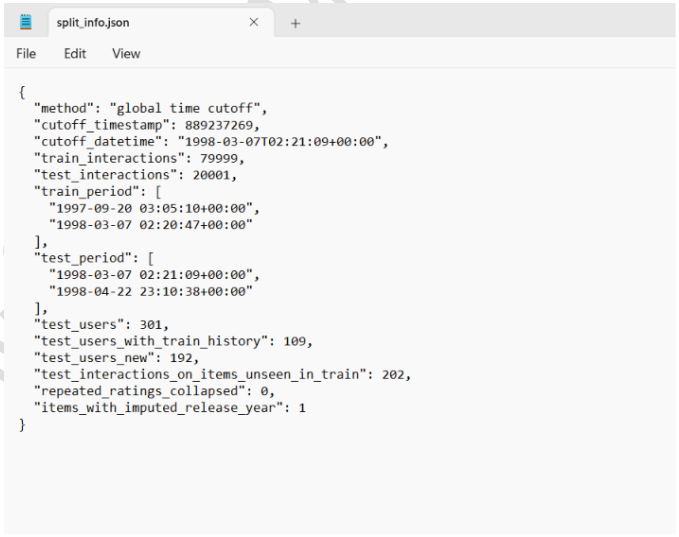


Figure A6. split_info.json: cutoff on 7 March 1998, 79,999 training and 20,001 test ratings

A.5 Stage 4: Data transformation

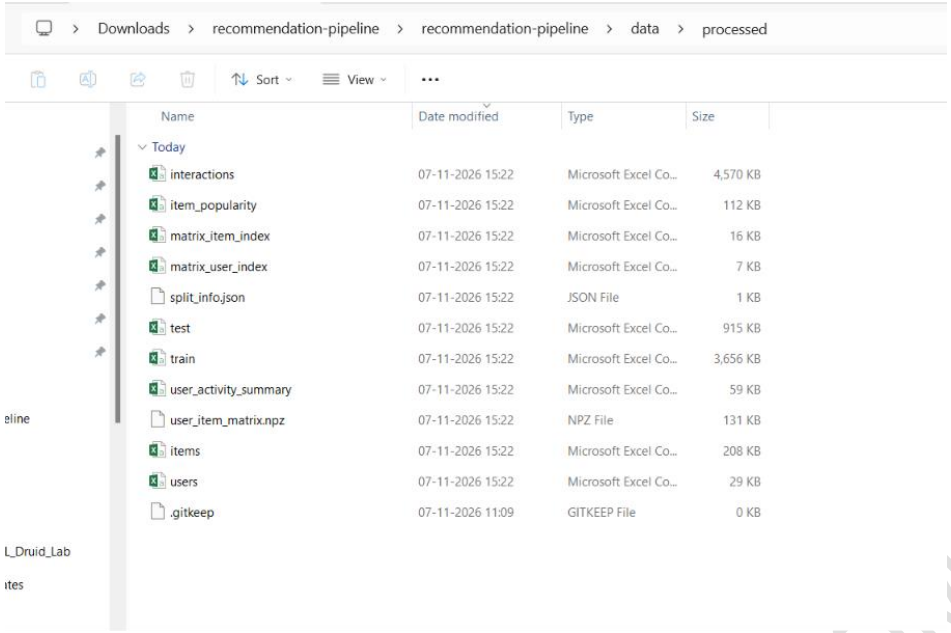
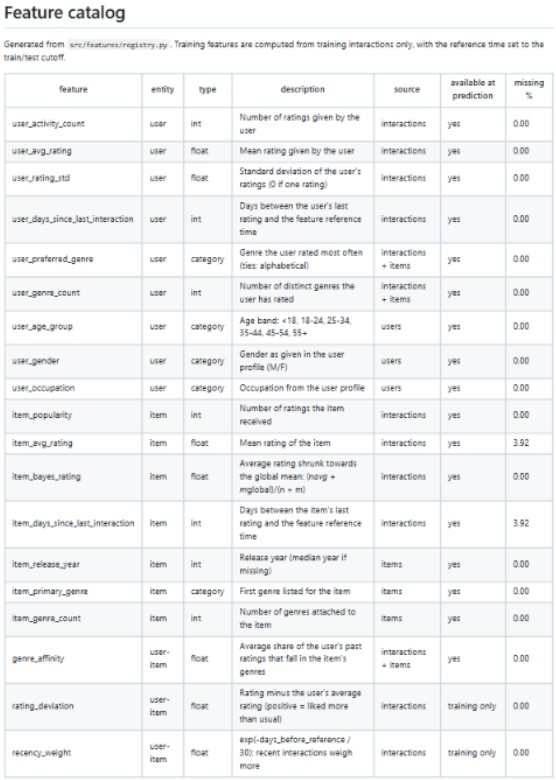


Figure A7. data/processed: cleaned tables, train/test files, user activity summary, item popularity and user-item matrix (NPZ)

A.6 Stage 5: Feature management



Feature catalog

Generated from `src/features/registry.py`. Training features are computed from training interactions only, with the reference time set to the train/test cutoff.

feature	entity	type	description	source	available at prediction	missing %
user_activity_count	user	int	Number of ratings given by the user	interactions	yes	0.00
user_avg_rating	user	float	Mean rating given by the user	interactions	yes	0.00
user_rating_std	user	float	Standard deviation of the user's ratings (0 if one rating)	interactions	yes	0.00
user_days_since_last_interaction	user	int	Days between the user's last rating and the feature reference time	interactions	yes	0.00
user_preferred_genre	user	category	Genre the user rated most often (ties: alphabetical)	interactions + items	yes	0.00
user_genre_count	user	int	Number of distinct genres the user has rated	interactions + items	yes	0.00
user_age_group	user	category	Age band: <18, 18-24, 25-34, 35-44, 45-54, 55+	users	yes	0.00
user_gender	user	category	Gender as given in the user profile (M/F)	users	yes	0.00
user_occupation	user	category	Occupation from the user profile	users	yes	0.00
item_popularity	item	int	Number of ratings the item received	interactions	yes	0.00
item_avg_rating	item	float	Mean rating of the item	interactions	yes	3.92
item_bayes_rating	item	float	Average rating shrunk towards the global mean: (novg + mglobal)/(n + m)	interactions	yes	0.00
item_days_since_last_interaction	item	int	Days between the item's last rating and the feature reference time	interactions	yes	3.92
item_release_year	item	int	Release year (median year if missing)	items	yes	0.00
item_primary_genre	item	category	First genre listed for the item	items	yes	0.00
item_genre_count	item	int	Number of genres attached to the item	items	yes	0.00
genre_affinity	user-item	float	Average share of the user's past ratings that fall in the item's genres	interactions + items	yes	0.00
rating_deviation	user-item	float	Rating minus the user's average rating (positive = liked more than usual)	interactions	training only	0.00
recency_weight	user-item	float	exp(-days_before_reference / 30): recent interactions weigh more	interactions	training only	0.00

Figure A8. Feature catalog generated from the feature registry (19 features)

2025AG05439 DMA

A.7 Stage 6: Model training and tracking (MLflow)

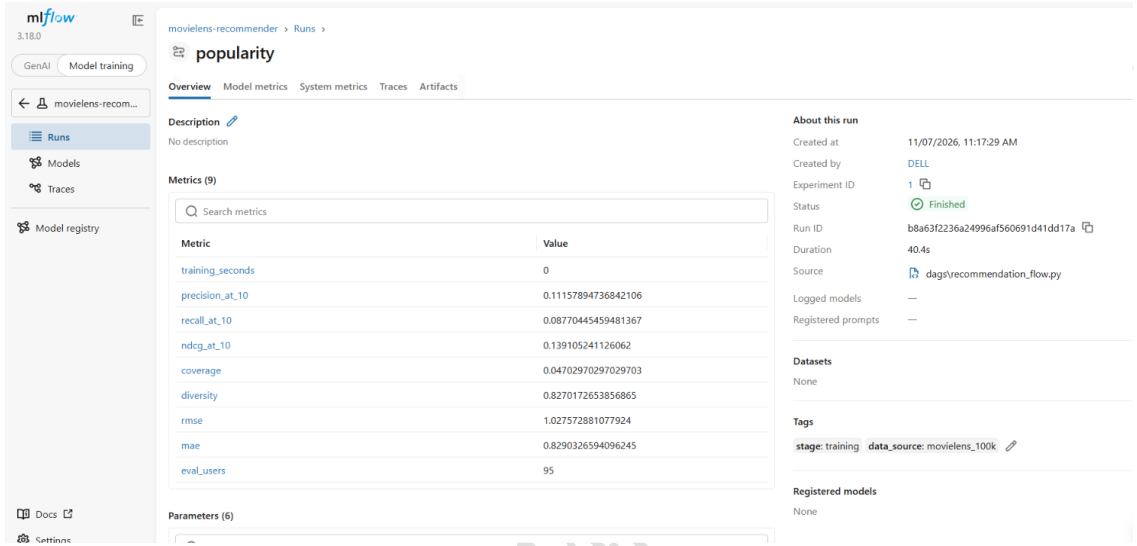


Figure A9. MLflow run for the popularity baseline: evaluation metrics

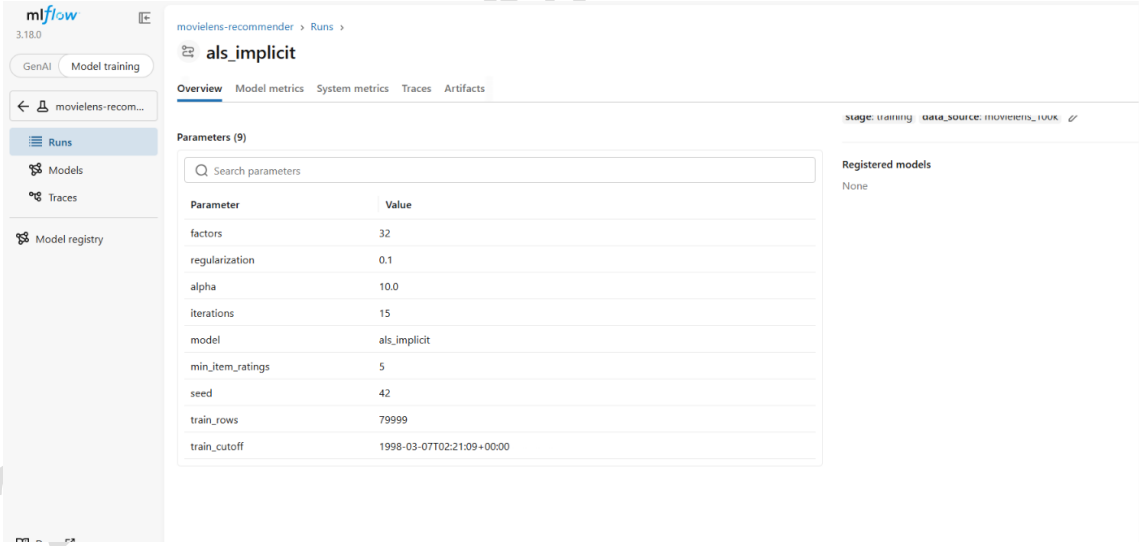


Figure A10. MLflow run for implicit ALS: logged parameters

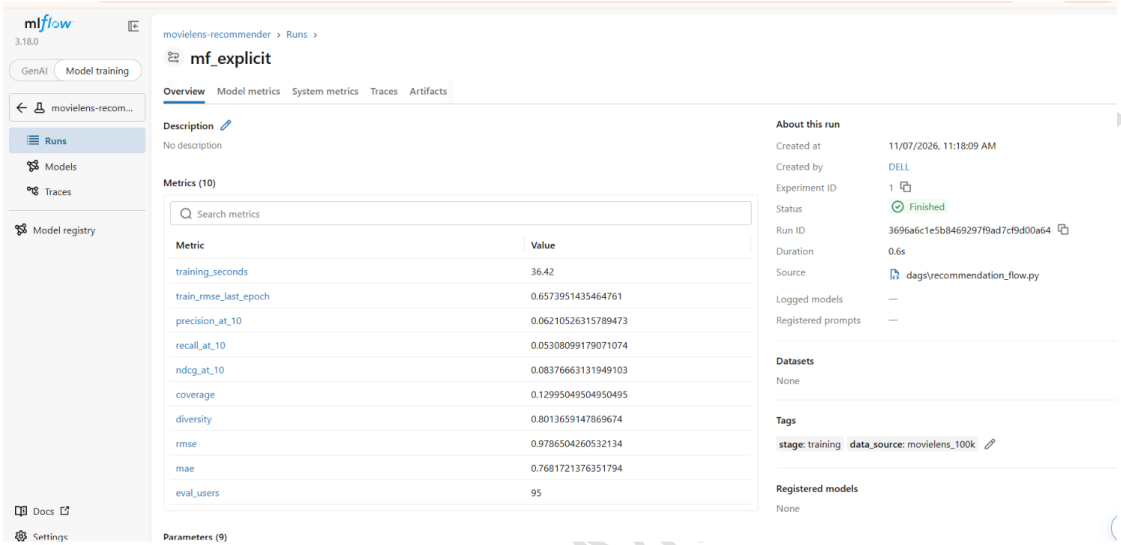


Figure A11. MLflow run for explicit MF: training time, training RMSE and evaluation metrics

A.8 Stage 7: Model evaluation

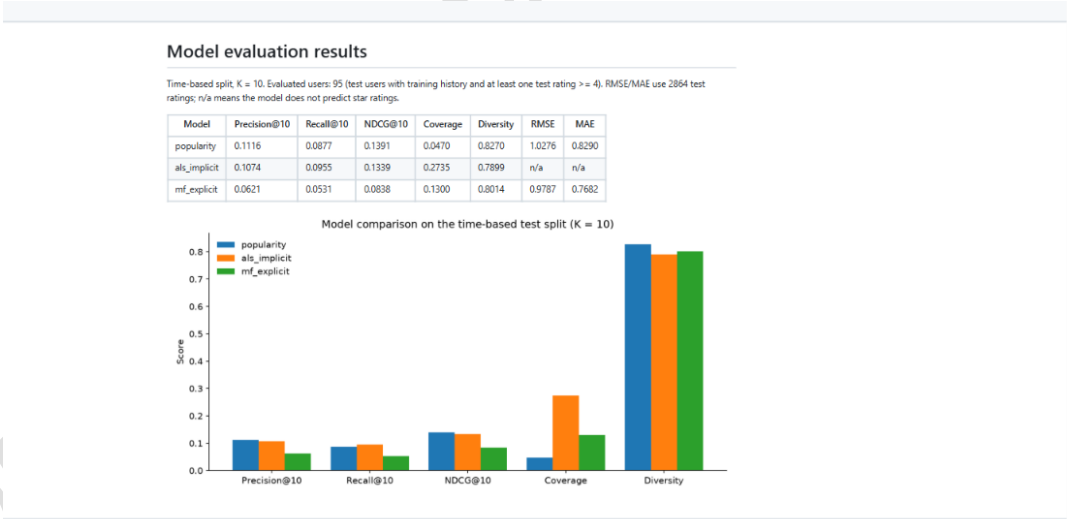


Figure A12. Model comparison table and chart (reports/model_results.md)

A.9 Orchestration (Prefect)

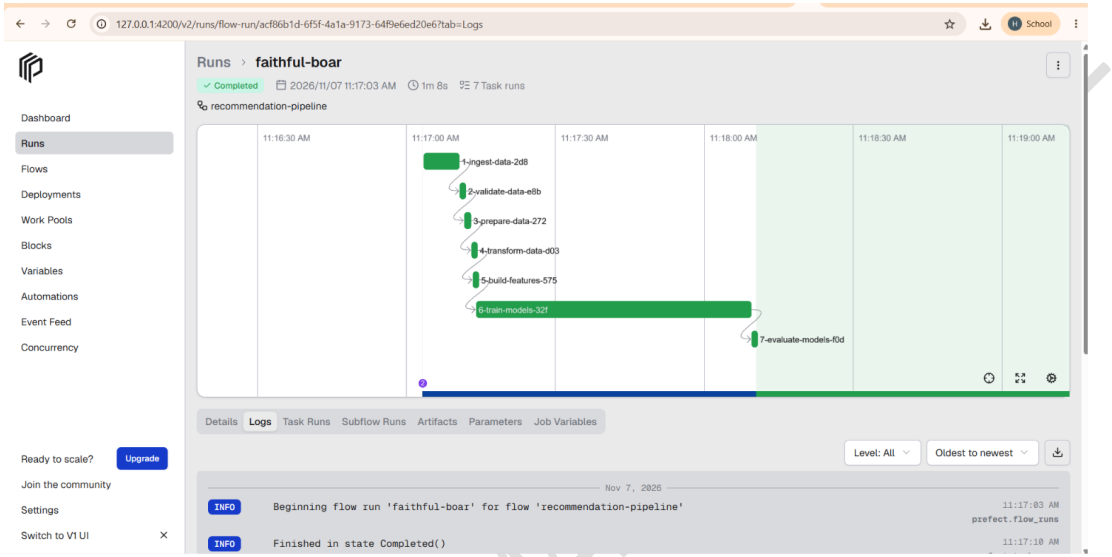


Figure A13. Prefect flow run 'faithful-boar': Completed in 1 min 8 s, seven tasks run in dependency order

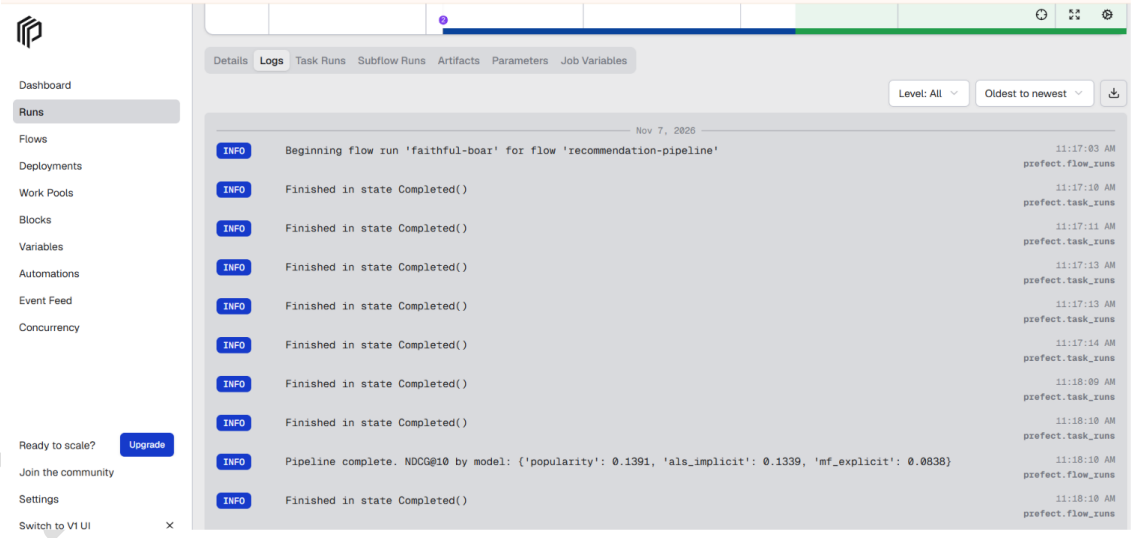


Figure A14. Prefect logs for the same run: every task Completed, then the NDCG@10 summary